

COMP544 — Emotion Classification Project Report

Emotion Classification from Text Using Classical Machine Learning

1. Introduction

Emotion classification is a fundamental natural language processing (NLP) task that involves assigning a discrete affective label to a piece of text. It has practical applications in social media monitoring, customer feedback analysis, and dialogue systems. Unlike binary sentiment analysis, fine-grained emotion classification presents additional challenges due to the semantic overlap between adjacent emotion categories and the inherently subjective nature of emotional expression.

This report describes the design, implementation, and evaluation of an emotion classification system trained on short text messages. The system explores four classical supervised learning algorithms combined with TF-IDF feature extraction. The central research question is: which combination of feature representation and classification algorithm yields the strongest generalisation performance on this multi-class task, particularly under varying amounts of training data?

2. Dataset Description

The dataset comprises 8,000 labelled text samples, each consisting of a short message and a corresponding emotion label drawn from seven categories: *sadness*, *neutral*, *worry*, *happiness*, *fun*, *surprise*, and *hate*. An initial exploratory analysis confirmed that the dataset contains no missing values or duplicate entries, indicating a clean and well-curated collection.

However, the class distribution is not uniform: the *surprise* category is underrepresented relative to the other six classes. This imbalance has methodological consequences for model evaluation, since a classifier that performs well on frequent classes but poorly on rare ones may appear deceptively strong under accuracy-based assessment. For this reason, macro-averaged F1 score — which weights all classes equally regardless of frequency — was adopted as a primary evaluation metric alongside standard accuracy.

3. Preprocessing Pipeline

Raw text cannot be consumed directly by classical machine learning classifiers and must first be normalised to reduce noise and improve vocabulary consistency. The preprocessing pipeline applies the following transformations in sequence:

1. **Lowercasing** — Converts all characters to lowercase to treat surface variants (e.g., *Happy* and *happy*) as identical tokens.
2. **URL removal** — Strips hyperlinks, which carry no semantic content relevant to emotion.
3. **Punctuation and number removal** — Eliminates non-alphabetic tokens that introduce sparsity without meaningful signal.
4. **Whitespace normalisation** — Collapses consecutive whitespace characters into a single space to standardise token boundaries.

These steps collectively ensure that the subsequent feature extraction stage operates on a clean, consistent vocabulary, which is especially important for controlling the dimensionality of sparse text representations.

4. Feature Extraction

Following preprocessing, text is transformed into numerical feature vectors using Term Frequency-Inverse Document Frequency (TF-IDF) weighting. TF-IDF assigns higher weight to terms that are frequent in a specific document but rare across the corpus, thereby emphasising discriminative vocabulary over common function words. Formally:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \log \frac{N}{1 + \text{DF}(t)}$$

where N is the total number of documents and $\text{DF}(t)$ is the number of documents containing term t .

The vectoriser was configured with the following settings, each motivated by empirical and theoretical considerations:

- `stop_words='english'` — Removes high-frequency function words (e.g., *the*, *is*, and *and*) that contribute little discriminative information.
- `ngram_range=(1, 2)` — Includes both unigrams and bigrams. Bigrams capture short phrases whose emotional meaning cannot be inferred from individual tokens alone (e.g., *"not happy"*, *"very worried"*).
- `max_features=10,000` — Caps the vocabulary at the 10,000 most informative terms to manage computational cost and reduce noise from extremely rare tokens.
- `sublinear_tf=True` — Applies a logarithmic transformation to raw term frequency, dampening the disproportionate influence of high-frequency terms within individual documents.
- `min_df=2` — Excludes tokens that appear in fewer than two documents, filtering out hapax legomena that are unlikely to generalise to unseen text.

A critical implementation constraint is that the TF-IDF vectoriser was fitted exclusively on the training partition of each experiment and subsequently applied to the test set using the learned vocabulary. Fitting on the full dataset before splitting would constitute data leakage and yield artificially optimistic evaluation results.

5. Machine Learning Models

Four supervised classification algorithms were evaluated, selected to provide broad coverage of different learning paradigms applicable to sparse text data.

Decision Tree. A decision tree recursively partitions the feature space by selecting the split that maximises information gain at each node. While decision trees are interpretable and computationally efficient, they are poorly suited to high-dimensional sparse inputs: with thousands of TF-IDF features, the tree is prone to overfitting individual idiosyncrasies of the training data rather than learning robust patterns. Balanced class weights were applied to partially compensate for class imbalance, and maximum depth was constrained to reduce overfitting.

Linear Support Vector Machine (LinearSVC). SVMs seek the hyperplane that maximises the margin between classes in the feature space. Linear SVMs are particularly well-suited to text classification because TF-IDF produces high-dimensional but linearly separable representations for many tasks; in such spaces, the linear kernel is both computationally efficient and empirically competitive with non-linear alternatives. Balanced class weights were used, and the regularisation parameter C was tuned to control the bias-variance trade-off.

k-Nearest Neighbours (kNN). kNN classifies a test instance by majority vote among its k nearest neighbours in the training set. Cosine distance was chosen as the similarity metric because it measures the angle between document vectors and is therefore invariant to document length — a more appropriate criterion than Euclidean distance for sparse text representations. Despite this, kNN is inherently disadvantaged in high-dimensional sparse spaces, where distances between points become increasingly uniform, degrading the quality of neighbourhood retrieval.

Multinomial Naïve Bayes. Naïve Bayes applies Bayes' theorem under a conditional independence assumption over features. Although this assumption is clearly violated in natural language (adjacent words are not independent), the model frequently achieves competitive performance on text classification tasks because the independence assumption, while incorrect, does not prevent reasonable posterior approximation in practice. The Laplace smoothing parameter α was tuned to prevent zero-probability estimates for unseen terms.

6. Experimental Setup

To assess the effect of training set size on model performance, three stratified train/test splits were evaluated: **40%/60%**, **30%/70%**, and **20%/80%**. Stratified splitting ensures that the proportional distribution of emotion labels is preserved across both partitions in each configuration, allowing fair comparison across splits. These proportions were chosen to simulate progressively data-scarce conditions, reflecting realistic settings where labelled data may be limited.

For each split, the TF-IDF vectoriser was re-fitted on the respective training partition only, ensuring that all reported results reflect genuine generalisation performance on unseen data.

7. Evaluation Metrics

Three complementary metrics were computed for each model and split configuration:

- **Accuracy** — The fraction of correctly classified instances. Provides an overall performance summary but can be misleading on imbalanced class distributions.
 - **Macro F1** — The unweighted mean of per-class F1 scores. By treating all classes equally, macro F1 penalises poor performance on minority classes such as *surprise*, making it the most informative metric for this dataset.
 - **Weighted F1** — A class-frequency-weighted mean of per-class F1 scores. Reflects aggregate performance while accounting for class support, and lies between accuracy and macro F1 in terms of sensitivity to class imbalance.
-

8. Results and Discussion

Across all three train/test split configurations, **LinearSVC achieved the highest performance**, consistent with the well-established effectiveness of linear classifiers on high-dimensional sparse text representations. The model benefited directly from the TF-IDF feature space, where class boundaries are approximately linear and the large feature dimensionality provides ample discriminative capacity.

Naïve Bayes served as the strongest baseline among the remaining models, performing reasonably well despite its independence assumption. This is consistent with prior literature demonstrating that probabilistic text classifiers can generalise effectively even when model assumptions are violated. **kNN** achieved moderate performance: cosine similarity partially offset the curse of dimensionality, but the model remained disadvantaged relative to the generative and discriminative alternatives. **Decision Tree** was the weakest model overall, which is expected given the mismatch between tree-based inductive bias and the structure of high-dimensional sparse feature spaces.

As the training set size decreased from 40% to 20%, all models exhibited a consistent decline in both accuracy and macro F1. This degradation was most pronounced in macro F1, highlighting the disproportionate impact of reduced data on minority-class performance. The *surprise* class, being underrepresented in the full dataset, was the most sensitive to training data scarcity.

Inspection of the confusion matrix for the best-performing model revealed that most predictions were correctly placed along the diagonal. The most common misclassification patterns occurred between semantically proximate categories: *sadness* and *worry* share overlapping linguistic markers of negative affect, while *happiness* and *fun* both involve positive valence with only subtle tonal differences. Such confusions are consistent with the inherent ambiguity of emotional expression in natural language and represent a fundamental challenge that classical bag-of-words representations cannot fully resolve.

9. Final Model

The final selected model is a pipeline consisting of **TF-IDF vectorisation followed by LinearSVC**. This combination was selected on the basis of its superior macro F1 performance across all evaluated split configurations, its robustness to class imbalance when using balanced class weights, and its computational efficiency relative to kernel-based or instance-based alternatives. The final model is trained on the full available training set and is used within the required `train_test(train_csv, test_csv)` function to generate emotion predictions for unseen test instances.

10. Conclusion

This project evaluated four classical machine learning approaches for multi-class emotion classification from short text messages. The experiments demonstrated that a TF-IDF representation combined with LinearSVC consistently outperforms Decision Tree, kNN, and Naïve Bayes across varying training set sizes. The results confirm the suitability of linear classifiers for high-dimensional sparse text data and highlight the importance of using macro F1 as the primary evaluation metric when class distributions are imbalanced. Future work could investigate transformer-based representations such as BERT, which capture contextual and semantic information that bag-of-words models cannot represent, and may yield substantially stronger performance on emotionally ambiguous instances.
